

```

231 .sticky .content-inner {
232   margin-bottom: 0px!important;
233   padding-bottom: 0px!important;
234   border-bottom: 0px!important;
235   -o-box-shadow: 0 1px 2px rgba(0,0,0,0.3);
236   -moz-box-shadow: 0 1px 2px rgba(0,0,0,0.3);
237   -webkit-box-shadow: 0 1px 2px rgba(0,0,0,0.3);
238   box-shadow: 0 1px 2px rgba(0,0,0,0.3);
239   background-color: #fff;
240   padding: 25px!important;
241   position: relative;

```

Aplicaciones web 1

Actividad 6

Actividad a desarrollar individualmente como práctica; y la entrega es individual para que quede el registro.

CSS

Nos solicitan estilizar el HTML realizado para nuestro comitente/cliente.

(Continuar con lo desarrollado en la actividad 5)

Antes de codificar, piense y escriba lo que va a realizar, es decir, un diseño mínimo, como colores a utilizar, tipografía, tamaños, etc. Esto nos permitirá saber qué es lo que quiero para luego buscar el cómo. **No se permite utilizar la IA para el desarrollo y planificación; solo se utilizará la IA al final para revisar lo hecho.**

Lo que se necesita es:

- Organizar proyecto y utilizar node y pnpm para trabajar:
 - Crear directorio de trabajo
 - Instalar pnpm (solo en el laboratorio - si trabajan con su notebook, ya debería estar instalado) -> `npm i -g pnpm`
 - Inicializar proyecto -> `pn init`
 - Instalar un paquete de servidor -> `pn add five-server`
 - Configurar scripts en package.json -> "dev": "five-server"
 - Crear archivo de configuración de five-server. `fiveserverrc`
- ```
{
```

**"root": "src"**

**}**

- Crear carpeta src y copiar proyecto de la actividad 5
- Correr proyecto -> **pn dev**
- Utilizar selectores de TIPO para estilizar los elementos de manera global.
- Utilizar selectores de CLASE para especificar los bloques principales de nuestro layout (plantilla). En el caso de que el desarrollo no cuente con los 3 bloques vistos (header, main y footer), deberá crearlos.
  - Los selectores de CLASE deben ser denominados con nombres acordes a la función de dicho bloque; esto mejora la lectura del código, el mantenimiento y escalado.
  - Utilizar selectores combinados.
- Utilizar el modelo de caja alternativo.
- Utilizar un reseteo básico y su organización.
- Por el momento, el uso de la IA SOLO será para verificar lo desarrollado. En el caso de utilizarla, incluir en este documento el historial de los prompts.

**Resolver aquí. Descargar en PDF y subir al repositorio.**

## Verificación de lo desarrollado

Prompt: Qué se podría mejorar en este archivo css?

Respuesta:

El CSS cumple lo pedido (reseteo, modelo de caja alternativo, selectores de tipo, de clase y combinados). Estos son los puntos que mejoraría, de más a menos importante:

### 1. Código repetido

- Los tres bloques de layout repiten `max-width: 960px; margin: 0 auto;`. Se puede agrupar:

CSS

```
.cabecera, .principal, .pie {
 max-width: 960px;
 margin: 0 auto;
}
```

Cada bloque conserva después solo lo propio (padding, fondo, borde).

- Los colores (`#1f3a5f`, `#2f6f8f`) y la fuente `Arial, Helvetica, sans-serif` aparecen en muchos lugares. Con variables CSS en `:root` (`--color-primario`, `--fuente-titulos`) un cambio de paleta se hace en un solo sitio. Si todavía no vieron variables en la materia, es opcional.

### 2. Código redundante o sin uso

- `html` se declara dos veces, y `font-size: 100%` ya es el valor por defecto.
- Las reglas de `hr` quedaron sin uso, porque saqué los `<hr>` del HTML. Lo mismo pasa con `h3` en los selectores de tipo.
- `hr` y `a` se estilan en el reseteo y otra vez en la sección global. Conviene dejar en el reseteo solo lo que se anula y el estilo real en la sección global.

### 3. Selectores frágiles

- `input, select, textarea { width: 100% }` también afectaría a checkboxes o radios si se agregan después. Es más seguro limitarlo a `input[type="text"], [type="email"]` y `[type="tel"]`, o excluir el botón con `input:not([type="submit"])`.
- Algunos selectores combinados suben la especificidad sin necesidad, como `.principal .formulario-contacto` (con `.formulario-contacto` alcanza). Para esta consigna sirven como ejemplo de combinados, pero en un proyecto real los evitaría.

### 4. Layout y responsive

- No hay ninguna media query. Todo es fluido, pero el `h1` de 2.25rem podría reducirse en pantallas chicas.
- En una página corta el pie queda pegado al contenido, no al borde inferior. Se resuelve con `body { min-height: 100vh; display: flex; flex-direction: column; }` y `.principal { flex: 1; }`.
- Los párrafos ocupan los 960px completos, lo que resulta largo de leer. Un `max-width: 65ch` en `p` mejora la lectura.

### 5. Accesibilidad

- Los enlaces dentro del texto (como el del pie) se distinguen solo por el color, sin subrayado hasta el hover. Es mejor mantener el subrayado y quitarlo solo en los botones de la navegación.
- Para el foco conviene `:focus-visible`, que muestra el contorno con teclado pero no al hacer clic con el mouse.

### 6. Nombres

- `.navegacion-principal` y `.principal` se parecen y pueden confundirse. Yo dejaría `.navegacion`.

## 7. Escalabilidad

- Si el sitio crece, conviene separar el reseteo, el layout y los componentes en archivos distintos dentro de `styles/` e importarlos desde `index.css`. Con dos páginas no hace falta.